

Digital Nurture 5.0 DeepSkilling

Spring REST using Spring Boot 3

Tasks:

1. Create a Spring Web Project using Maven

SpringLearnApplication.java

```
package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);

    public static void main(String[] args) {

        LOGGER.info("Spring Boot Application Started");

        SpringApplication.run(SpringLearnApplication.class, args);

        LOGGER.info("Application Running Successfully");
    }
}
```

Output

```
2026-06-30T18:53:28.701+05:30 INFO 32844 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-06-30T18:53:28.701+05:30 INFO 32844 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.80]
2026-06-30T18:53:28.742+05:30 INFO 32844 --- [spring-learn] [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-06-30T18:53:28.742+05:30 INFO 32844 --- [spring-learn] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed
2026-06-30T18:53:29.061+05:30 INFO 32844 --- [spring-learn] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 32844
2026-06-30T18:53:29.093+05:30 INFO 32844 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http)
2026-06-30T18:53:29.103+05:30 INFO 32844 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Started SpringLearnApplication in 1.106 s
2026-06-30T18:53:29.106+05:30 INFO 32844 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Application Running Successfully
```

2. Spring Core – Load SimpleDateFormat from Spring Configuration XML

Date-format.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat"
          class="java.text.SimpleDateFormat">

        <constructor-arg value="dd/MM/yyyy"/>

    </bean>

</beans>
```

SpringLearnApplication.java

```
package com.cognizant.springlearn;

import java.text.SimpleDateFormat;
import java.util.Date;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

@SpringBootApplication
public class SpringLearnApplication {

    public static void main(String[] args) throws Exception {

        SpringApplication.run(SpringLearnApplication.class, args);

        displayDate();
    }

    public static void displayDate() throws Exception {

        ApplicationContext context =
            new ClassPathXmlApplicationContext("date-format.xml");

        SimpleDateFormat format =
            context.getBean("dateFormat", SimpleDateFormat.class);

        Date date = format.parse("31/12/2018");

        System.out.println(date);
    }
}
```

Output

```
2026-06-30T19:02:53.670+05:30 INFO 41304 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache T
2026-06-30T19:02:53.710+05:30 INFO 41304 --- [spring-learn] [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebAp
2026-06-30T19:02:53.711+05:30 INFO 41304 --- [spring-learn] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initia
2026-06-30T19:02:54.155+05:30 INFO 41304 --- [spring-learn] [ restartedMain] o.s.b.d.a.OptionalliveReloadServer : LiveReload server is running on po
2026-06-30T19:02:54.197+05:30 INFO 41304 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http)
2026-06-30T19:02:54.210+05:30 INFO 41304 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Started SpringLearnApplication in
Mon Dec 31 00:00:00 IST 2018
```

3. Hello World RESTful Web Service

HelloController.java

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(HelloController.class);

    @GetMapping("/hello")
    public String sayHello() {

        LOGGER.info("START - sayHello()");

        String message = "Hello World!!";

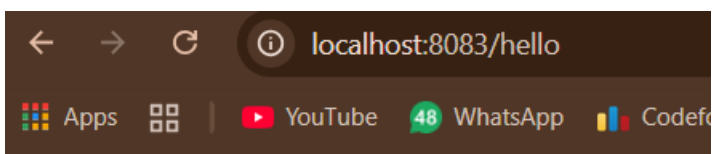
        LOGGER.info("END - sayHello()");

        return message;
    }
}
```

Application.properties

```
spring.application.name=spring-learn
server.port=8083
```

Output



Hello World!!

4. REST - Country Web Service

Country.java

```
package com.cognizant.springlearn.model;

public class Country {

    private String code;
    private String name;

    public Country() {
    }

    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

Country.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="country"
        class="com.cognizant.springlearn.model.Country">

        <property name="code" value="IN"/>
        <property name="name" value="India"/>

    </bean>

</beans>
```

CountryController.java

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.springlearn.model.Country;

@RestController
public class CountryController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);

    @RequestMapping("/country")
    public Country getCountryIndia() {

        LOGGER.info("START - getCountryIndia()");

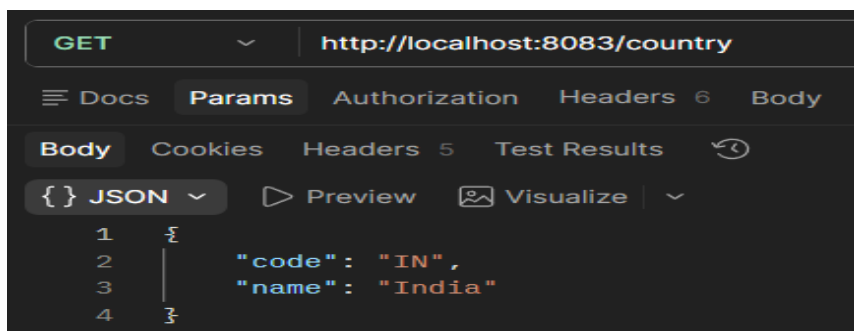
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");

        Country country =
            context.getBean("country", Country.class);

        LOGGER.info("END - getCountryIndia()");

        return country;
    }
}
```

Output



```
2026-06-30T20:30:18.762+05:30 INFO 41224 --- [spring-learn] [nio-8083-exec-1] c.c.s.controller.CountryController : START - getCountryIndia()
2026-06-30T20:30:18.803+05:30 INFO 41224 --- [spring-learn] [nio-8083-exec-1] c.c.s.controller.CountryController : END - getCountryIndia()
2026-06-30T20:30:52.210+05:30 INFO 41224 --- [spring-learn] [nio-8083-exec-3] c.c.s.controller.CountryController : START - getCountryIndia()
2026-06-30T20:30:52.231+05:30 INFO 41224 --- [spring-learn] [nio-8083-exec-3] c.c.s.controller.CountryController : END - getCountryIndia()
```

5. REST - Get country based on country code

Country.xml

```
<bean id="countryList" class="java.util.ArrayList">

    <constructor-arg>
        <list>

            <bean class="com.cognizant.springlearn.model.Country">
                <property name="code" value="IN"/>
                <property name="name" value="India"/>
            </bean>

            <bean class="com.cognizant.springlearn.model.Country">
                <property name="code" value="US"/>
                <property name="name" value="United States"/>
            </bean>

            <bean class="com.cognizant.springlearn.model.Country">
                <property name="code" value="JP"/>
                <property name="name" value="Japan"/>
            </bean>

            <bean class="com.cognizant.springlearn.model.Country">
                <property name="code" value="AU"/>
                <property name="name" value="Australia"/>
            </bean>

        </list>
    </constructor-arg>

</bean>

</beans>
```

CountryController.java

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.springlearn.model.Country;
import com.cognizant.springlearn.service.CountryService;

@RestController
public class CountryController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);

    @Autowired
    private CountryService countryService;

    @GetMapping("/countries/{code}")
    public Country getCountry(@PathVariable String code) {

        LOGGER.info("START - getCountry()");

        Country country = countryService.getCountry(code);

        LOGGER.info("END - getCountry()");

        return country;
    }
}
```

CountryService.java

```
package com.cognizant.springlearn.service;

import java.util.List;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;

import com.cognizant.springlearn.model.Country;

@Service
public class CountryService {

    public Country getCountry(String code) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");

        List<Country> countries =
            (List<Country>) context.getBean("countryList");

        for (Country country : countries) {

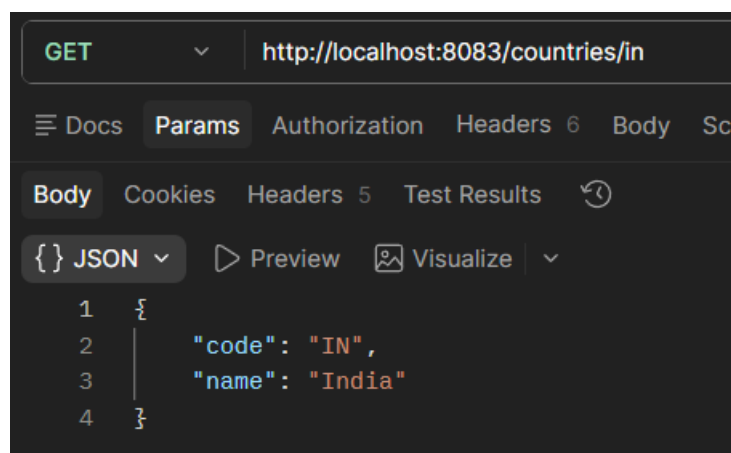
            if (country.getCode().equalsIgnoreCase(code)) {
                return country;
            }

        }

        return null;
    }

}
```

Output



GET http://localhost:8083/countries/in

Docs Params Authorization Headers 6 Body Cookies

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "code": "IN",
3   "name": "India"
4 }
```

6. Create authentication service that returns JWT

AuthenticationController.java

```
package com.cognizant.springlearn.controller;

import java.util.HashMap;
import java.util.Map;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AuthenticationController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(AuthenticationController.class);

    @GetMapping("/authenticate")
    public Map<String, String> authenticate(
        @RequestHeader("Authorization") String authHeader) {

        LOGGER.info("START");

        LOGGER.debug("Authorization Header : {}", authHeader);

        Map<String, String> map = new HashMap<>();

        map.put("token", "");

        LOGGER.info("END");

        return map;
    }
}
```

SecurityConfig.java

```
package com.cognizant.springlearn.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public InMemoryUserDetailsManager userDetailsService() {

        UserDetails admin = User.builder()
            .username("admin")
            .password(passwordEncoder().encode("pwd"))
            .roles("ADMIN")
            .build();

        UserDetails user = User.builder()
            .username("user")
            .password(passwordEncoder().encode("pwd"))
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(admin, user);
    }
}
```



```

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http
        .csrf(csrf -> csrf.disable())
        .httpBasic(Customizer.withDefaults())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
            .requestMatchers("/countries").hasRole("USER")
            .anyRequest().authenticated());

    return http.build();
}

```

Application.properties

```

spring.application.name=spring-learn
server.port=8090

logging.level.com.cognizant=DEBUG

```

Output

